

# 实验一 从 0 手写 attention

2023112451 顾雨航

## 一、实验目的与任务说明

本实验的目标是基于自定义 Multi-Head Attention (MHA) 层，构建一个疾病分类模型，并完成训练、测试和结果分析。实验数据来自 train\_set.json 和 test\_set.json，需要先对病历中的症状信息进行处理，将其转换为模型可输入的整数索引序列，并通过截断和填充统一长度。

在模型实现中，实验要求手写 ScaledDotProductAttention 类，并在其基础上实现 MultiHeadAttention，进一步搭建包含多头注意力、残差连接、LayerNorm 和前馈网络的简易 Transformer Encoder，最后接全连接层完成疾病分类。同时，模型还需要使用 Padding Mask，避免注意力分配到无效的 [PAD] 位置。

除分类任务外，本实验还需要完成两部分分析工作：一是提取样本的 Attention Map，并用热力图展示医学症状之间的关联；二是比较使用缩放因子  $1/\sqrt{d_k}$  与不使用缩放因子时 Loss 曲线的差异，从而分析缩放操作对模型训练稳定性的影响。

## 二、数据集与预处理

本实验使用 train\_set.json 和 test\_set.json 作为数据集。每条样本主要包含显性症状 exp\_sxs、隐性症状 imp\_sxs 以及疾病标签 label。其中，症状以键值对形式存储，值为 1 表示症状存在，值为 0 表示症状不存在。

在数据预处理阶段，首先将每条病历中的症状信息整理为 token 序列。对于值为 1 的症状，直接保留其名称；对于值为 0 的症状，可以根据需要添加前缀“无”后再保留，以帮助模型学习阴性症状信息。随后，根据训练集构建词表，将每个 token 映射为对应的整数索引，同时将疾病类别映射为整数标签。

```
def record_to_tokens(record: Dict, include_neg: bool = True) -> List[str]:
    """
    将一条病历转为“单词序列”。
    - 对值为 1 的症状直接保留；
    - include_neg=True 时，对值为 0 的症状加前缀“无”保留，可帮助模型感知缺失信息。
    """
    tokens: List[str] = []
    for k, v in record["exp_sxs"].items():
        tokens.append(k if v == "1" else f"无{k}" if include_neg else None)
    for k, v in record["imp_sxs"].items():
        tokens.append(k if v == "1" else f"无{k}" if include_neg else None)
    return [t for t in tokens if t is not None]
```

为了适配模型输入，所有样本序列都需要统一到固定长度。本实验将序列截断或填充到 max\_len=128，并同步生成 mask：真实 token 位置记为 1，填充位置记为 0。该 mask 会在后续注意力计算中作为 Padding Mask 使用，避免模型关注无意义的填充内容。

```

class SymptomDataset(Dataset):
    """
    将 JSON 数据封装为 PyTorch Dataset, 并完成:
    - 构建词表
    - 将样本转为定长 token id 序列与 mask
    """
    def __init__(
        self,
        path: str,
        vocab: Vocabulary = None,
        label2id: Dict[str, int] = None,
        max_len: int = 128,
        include_neg: bool = True,
    ):
        super().__init__()
        with open(path, "r", encoding="utf8") as f:
            self.raw = json.load(f)
        self.max_len = max_len
        self.include_neg = include_neg

        # 标签映射: 若未传入则按字典序创建
        if label2id is None:
            labels = sorted({r["label"] for r in self.raw})
            self.label2id = {lb: i for i, lb in enumerate(labels)}
        else:
            self.label2id = label2id
        self.id2label = {v: k for k, v in self.label2id.items()}

        # 词表: 若未传入则用全量数据构建
        self.vocab = vocab or Vocabulary()
        if vocab is None:
            for r in self.raw:
                for t in record_to_tokens(r, include_neg=self.include_neg):
                    self.vocab.add_token(t)

        # 预编码样本
        self.samples: List[Tuple[torch.Tensor, torch.Tensor, torch.Tensor]] = []
        for r in self.raw:
            ids, mask = self.encode_tokens(record_to_tokens(r, include_neg=self.include_neg))
            label_id = self.label2id.get(r["label"], -1)
            self.samples.append((ids, mask, torch.tensor(label_id, dtype=torch.long)))

    def encode_tokens(self, tokens: List[str]) -> Tuple[torch.Tensor, torch.Tensor]:
        """将 token 列表转为定长 id 序列与 mask。"""
        ids = [self.vocab.encode(t) for t in tokens][: self.max_len]
        mask = [1] * len(ids)
        pad_len = self.max_len - len(ids)
        if pad_len > 0:
            ids += [0] * pad_len
            mask += [0] * pad_len
        return torch.tensor(ids, dtype=torch.long), torch.tensor(mask, dtype=torch.float32)

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, idx: int):
        return self.samples[idx]

```

### 三、模型设计与关键实现

本实验模型基于 Transformer Encoder 的基本结构搭建, 整体由词嵌入层、位置编码层、Multi-Head Attention、前馈神经网络以及分类层组成。输入的症状序列首先经过 Embedding 映射为稠密向量, 再叠加位置编码, 使模型能够感知序列中的位置信息。随后, 特征进入自定义的注意力模块进行特征交互, 最后通过池化和全连接层输出疾病类别。

在注意力实现部分, 实验首先手写了 ScaledDotProductAttention 类。该模块

通过计算 Q 与 K 的点积得到注意力分数，并在默认情况下除以  $\sqrt{d_k}$  进行缩放，以避免数值过大导致 softmax 过于尖锐。之后再结合 Padding Mask，将填充位置置为极小值，使这些位置在注意力分布中接近 0，最后与 V 相乘得到输出结果。这一部分是实验要求中的核心手写算子。

在此基础上，进一步实现了 MultiHeadAttention 模块。模型先对输入进行线性变换，生成 Q、K 和 V，再将其拆分为多个注意力头，并分别调用前面实现的 ScaledDotProductAttention 进行计算。各头输出拼接后，再经过线性层映射回原始维度，从而得到多头注意力结果。与单头注意力相比，多头结构能够从不同子空间学习症状之间的关联信息，提高特征表达能力。

为了完成分类任务，本实验在多头注意力后加入残差连接、LayerNorm 和前馈网络，构成一个简化的 Transformer Encoder 层。随后，对非 Padding 位置进行平均池化，并通过全连接层输出最终的疾病类别概率。整个模型在保留 Transformer 核心思想的同时，结构相对简洁，适合本次医学文本分类实验。

下面是手写缩放点积注意力和手写多头注意力代码：

```
class ScaledDotProductAttention(nn.Module):
    """
    手写缩放点积注意力:
    - 可通过 use_scale 控制是否除以  $\sqrt{d_k}$ 
    - mask 为 0 的位置填充 -1e9, 确保 Softmax 后权重接近 0
    """
    def __init__(self, dropout: float = 0.1, use_scale: bool = True):
        super().__init__()
        self.dropout = nn.Dropout(dropout)
        self.use_scale = use_scale

    def forward(
        self,
        Q: torch.Tensor,
        K: torch.Tensor,
        V: torch.Tensor,
        mask: torch.Tensor = None,
    ) -> Tuple[torch.Tensor, torch.Tensor]:
        """
        Q, K, V: [batch, head, seq_len, d_k]
        mask: [batch, 1, 1, seq_len], 1 表示有效, 0 表示 PAD
        """
        d_k = Q.size(-1)
        scores = torch.matmul(Q, K.transpose(-2, -1)) # [B, h, L, L]
        if self.use_scale:
            scores = scores / math.sqrt(d_k)
        if mask is not None:
            scores = scores.masked_fill(mask == 0, -1e9)
        attn = torch.softmax(scores, dim=-1)
        attn = self.dropout(attn)
        output = torch.matmul(attn, V) # [B, h, L, d_k]
        return output, attn
```



```

class MultiHeadAttention(nn.Module):
    """
    多头注意力，完全调用上面的 ScaledDotProductAttention。
    """
    def __init__(self, d_model: int, num_heads: int, dropout: float = 0.1, use_scale: bool = True):
        super().__init__()
        assert d_model % num_heads == 0, "d_model 必须能被 num_heads 整除"
        self.num_heads = num_heads
        self.d_k = d_model // num_heads
        # debug_shapes 用于维度追踪绘图
        self.debug_shapes: Dict[str, Tuple[int, ...]] = {}

        self.W_Q = nn.Linear(d_model, d_model)
        self.W_K = nn.Linear(d_model, d_model)
        self.W_V = nn.Linear(d_model, d_model)
        self.fc = nn.Linear(d_model, d_model)
        self.attention = ScaledDotProductAttention(dropout=dropout, use_scale=use_scale)
        self.dropout = nn.Dropout(dropout)

    def forward(
        self,
        query: torch.Tensor,
        key: torch.Tensor,
        value: torch.Tensor,
        mask: torch.Tensor = None,
        return_attn: bool = False,
    ) -> Tuple[torch.Tensor, torch.Tensor]:
        B, L, _ = query.size()

        # 线性变换后分头
        Q = self.W_Q(query).view(B, L, self.num_heads, self.d_k).transpose(1, 2)
        K = self.W_K(key).view(B, L, self.num_heads, self.d_k).transpose(1, 2)
        V = self.W_V(value).view(B, L, self.num_heads, self.d_k).transpose(1, 2)

        # 维度追踪：记录进入 MHA 前 / 分头后形状
        self.debug_shapes["before_attn"] = tuple(query.size()) # [B, L, d_model]
        self.debug_shapes["after_split"] = tuple(Q.size()) # [B, num_heads, L, d_k]

        # 缩放点积注意力
        context, attn = self.attention(Q, K, V, mask)

        # 拼接各头
        context = context.transpose(1, 2).contiguous().view(B, L, self.num_heads * self.d_k)
        self.debug_shapes["after_concat"] = tuple(context.size()) # [B, L, d_model]
        out = self.fc(context)
        out = self.dropout(out)
        return (out, attn) if return_attn else (out, None)

```

## 四、实验设置与训练过程

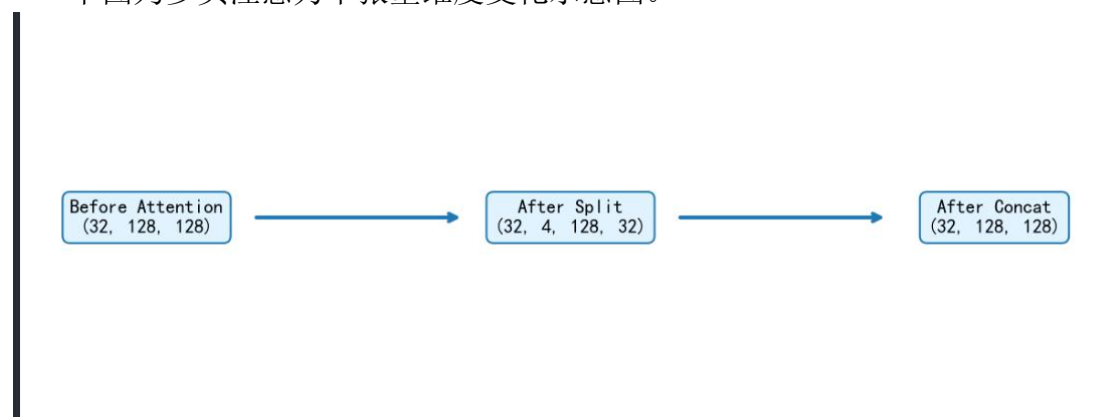
本实验在 PyTorch 框架下完成，模型训练和测试均基于 train\_set.json 与 test\_set.json 数据集进行。实验中采用的主要参数包括：序列最大长度 max\_len=128，批大小 batch\_size=32，隐藏层维度 d\_model=128，注意力头数 num\_heads=4，前馈网络维度 d\_ff=256，Dropout 比例 dropout=0.1，学习率 lr=2e-3。训练过程中，先将训练集按 9:1 划分为训练集和验证集，再在测试集上进行最终评估。

模型训练时，采用交叉熵损失函数作为优化目标，并使用 Adam 优化器更新参数。在每一轮训练中，程序会分别计算训练集和验证集上的 Loss、Accuracy 和 F1-score，用于观察模型的收敛情况和泛化能力。同时，程序会保存每轮训练得到的 Loss 与 F1 数据，并自动生成曲线图，便于后续进行实验结果分析。

为了更直观地展示多头注意力内部的数据流，本实验还对张量在注意力计算过程中的维度变化进行了记录与可视化。具体来说，输入特征进入注意力模块前的维度为 [B, L, d\_model]，在 Split 头之后变为 [B, num\_heads, L, d\_k]，最后在

各头拼接完成后恢复为 [B, L, d\_model]。这一过程说明，多头注意力本质上是先将特征拆分到多个子空间分别建模，再将多头结果重新整合，以获得更丰富的表示能力。

下图为多头注意力中张量维度变化示意图。



## 五、实验结果与分析

本章主要从缩放因子对训练过程的影响以及注意力可视化两个方面对实验结果进行分析。训练结果截图如下：

```
Anaconda Prompt
(base) D:\>cd D:\A2026CSLAB\LLM_lab\LAB1

(base) D:\A2026CSLAB\LLM_lab\LAB1>conda activate nlp1ab

(nlp1ab) D:\A2026CSLAB\LLM_lab\LAB1>python mha_classifier.py --train train_set.json --test test_set.json --epochs 15 --m
ax_len 128 --visualize --plot_shapes
设备: cuda | 词表大小: 195 | 类别数: 4
使用缩放因子: True
Epoch 01 | Train Loss 0.8699 Acc 0.6156 F1 0.5638 || Val Loss 0.6893 Acc 0.7150 F1 0.7189
Epoch 02 | Train Loss 0.6374 Acc 0.7305 F1 0.6983 || Val Loss 0.6816 Acc 0.6700 F1 0.6835
Epoch 03 | Train Loss 0.5999 Acc 0.7438 F1 0.7103 || Val Loss 0.6567 Acc 0.6363 F1 0.6003
Epoch 04 | Train Loss 0.6055 Acc 0.7297 F1 0.6887 || Val Loss 0.6527 Acc 0.6500 F1 0.6398
Epoch 05 | Train Loss 0.5364 Acc 0.7703 F1 0.7304 || Val Loss 0.6313 Acc 0.7150 F1 0.6939
Epoch 06 | Train Loss 0.5433 Acc 0.7648 F1 0.7417 || Val Loss 0.6391 Acc 0.7150 F1 0.7139
Epoch 07 | Train Loss 0.5300 Acc 0.7820 F1 0.7473 || Val Loss 0.6676 Acc 0.7150 F1 0.7236
Epoch 08 | Train Loss 0.4814 Acc 0.8023 F1 0.7670 || Val Loss 0.6757 Acc 0.6700 F1 0.6541
Epoch 09 | Train Loss 0.5097 Acc 0.7648 F1 0.7494 || Val Loss 0.6535 Acc 0.6937 F1 0.6850
Epoch 10 | Train Loss 0.4931 Acc 0.7758 F1 0.7497 || Val Loss 0.6361 Acc 0.7150 F1 0.6770
Epoch 11 | Train Loss 0.4698 Acc 0.7984 F1 0.7787 || Val Loss 0.7084 Acc 0.6575 F1 0.6422
Epoch 12 | Train Loss 0.4552 Acc 0.7992 F1 0.7631 || Val Loss 0.7373 Acc 0.6763 F1 0.6788
Epoch 13 | Train Loss 0.4166 Acc 0.8313 F1 0.8042 || Val Loss 0.6560 Acc 0.6712 F1 0.6571
Epoch 14 | Train Loss 0.4130 Acc 0.8289 F1 0.7945 || Val Loss 0.7335 Acc 0.6825 F1 0.6607
Epoch 15 | Train Loss 0.3933 Acc 0.8453 F1 0.8106 || Val Loss 0.6346 Acc 0.6888 F1 0.6905
[TEST] Loss 0.7320 Acc 0.6933 F1 0.6528
训练/验证 Loss 与 F1 曲线数据已写入 loss_history.json, 可用于报告绘图对比。
Loss/F1 曲线已保存到 loss_curve.png
注意力图已保存到 attention.png
维度追踪: {'before_attn': (32, 128, 128), 'after_split': (32, 4, 128, 32), 'after_concat': (32, 128, 128)}
维度变化图已保存到 tensor_shapes.png
```

```
Anaconda Prompt
训练/验证 Loss 与 F1 曲线数据已写入 loss_history.json, 可用于报告绘图对比。
Loss/F1 曲线已保存到 loss_curve.png
注意力图已保存到 attention.png
维度追踪: {'before_attn': (32, 128, 128), 'after_split': (32, 4, 128, 32), 'after_concat': (32, 128, 128)}
维度变化图已保存到 tensor_shapes.png

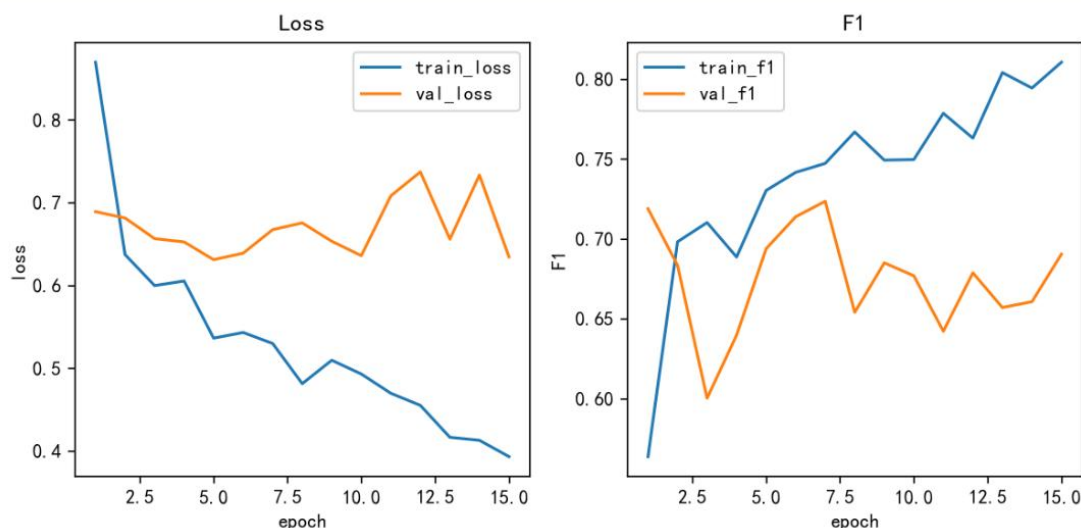
(nlp1ab) D:\A2026CSLAB\LLM_lab\LAB1>python mha_classifier.py --train train_set.json --test test_set.json --epochs 15 --m
ax_len 128 --no_scale
设备: cuda | 词表大小: 195 | 类别数: 4
使用缩放因子: False
Epoch 01 | Train Loss 0.8478 Acc 0.6195 F1 0.5686 || Val Loss 0.7171 Acc 0.6500 F1 0.6172
Epoch 02 | Train Loss 0.6617 Acc 0.7234 F1 0.6878 || Val Loss 0.7320 Acc 0.6687 F1 0.6470
Epoch 03 | Train Loss 0.6138 Acc 0.7125 F1 0.6923 || Val Loss 0.7419 Acc 0.6413 F1 0.6026
Epoch 04 | Train Loss 0.6356 Acc 0.7281 F1 0.6844 || Val Loss 0.6737 Acc 0.7337 F1 0.7399
Epoch 05 | Train Loss 0.5688 Acc 0.7609 F1 0.7172 || Val Loss 0.6198 Acc 0.7163 F1 0.7007
Epoch 06 | Train Loss 0.5628 Acc 0.7594 F1 0.7342 || Val Loss 0.6582 Acc 0.7538 F1 0.7475
Epoch 07 | Train Loss 0.5658 Acc 0.7648 F1 0.7237 || Val Loss 0.6605 Acc 0.6775 F1 0.6808
Epoch 08 | Train Loss 0.5347 Acc 0.7555 F1 0.7238 || Val Loss 0.6450 Acc 0.6763 F1 0.6631
Epoch 09 | Train Loss 0.5778 Acc 0.7484 F1 0.7313 || Val Loss 0.6492 Acc 0.6837 F1 0.6902
Epoch 10 | Train Loss 0.5820 Acc 0.7453 F1 0.7028 || Val Loss 0.6363 Acc 0.6900 F1 0.6896
Epoch 11 | Train Loss 0.5254 Acc 0.7578 F1 0.7409 || Val Loss 0.6291 Acc 0.6700 F1 0.6498
Epoch 12 | Train Loss 0.5091 Acc 0.7766 F1 0.7417 || Val Loss 0.6968 Acc 0.7013 F1 0.6963
Epoch 13 | Train Loss 0.4960 Acc 0.7891 F1 0.7516 || Val Loss 0.6844 Acc 0.6638 F1 0.6431
Epoch 14 | Train Loss 0.4805 Acc 0.7898 F1 0.7528 || Val Loss 0.6795 Acc 0.7025 F1 0.7072
Epoch 15 | Train Loss 0.4659 Acc 0.7906 F1 0.7551 || Val Loss 0.6205 Acc 0.6950 F1 0.6926
[TEST] Loss 0.7069 Acc 0.6969 F1 0.6640
训练/验证 Loss 与 F1 曲线数据已写入 loss_history_no_scale.json, 可用于报告绘图对比。
Loss/F1 曲线已保存到 loss_curve_no_scale.png

(nlp1ab) D:\A2026CSLAB\LLM_lab\LAB1>
```

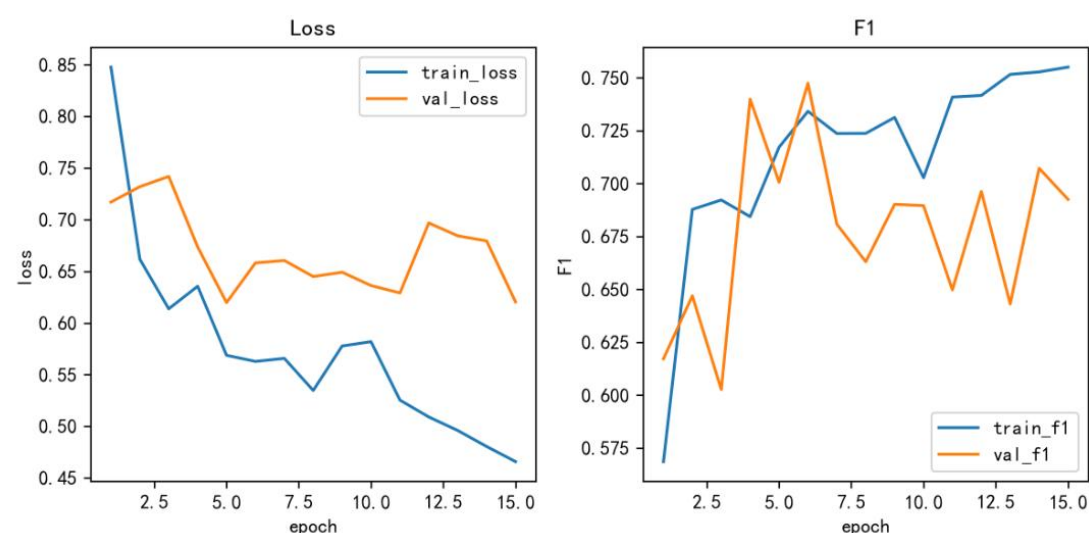
## 5.1 缩放因子影响

为了分析缩放因子  $1/\sqrt{d_k}$  的作用，本实验分别进行了“使用缩放因子”和“不使用缩放因子”两组实验，并记录了训练过程中的 Loss 变化曲线。

下图为使用缩放因子的 loss 曲线：



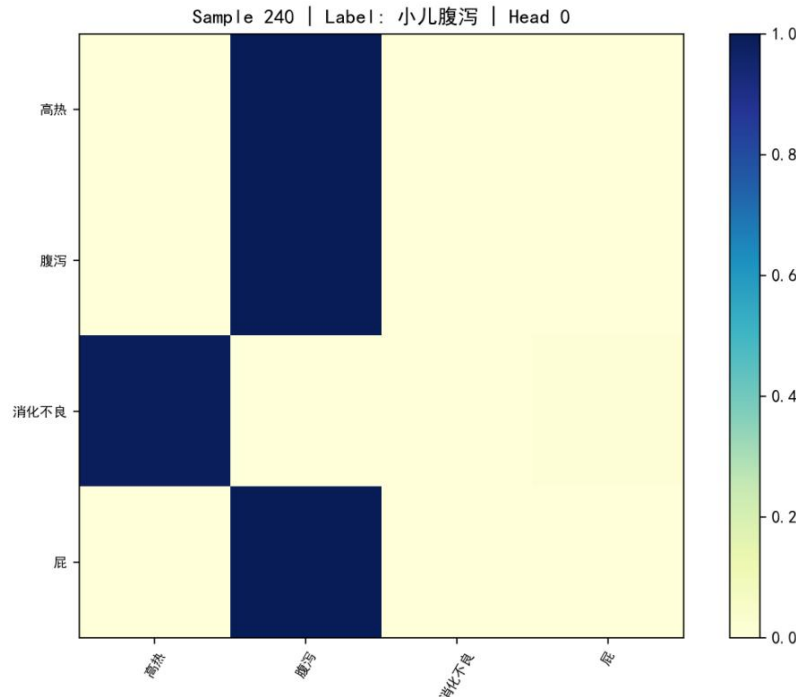
下图为不使用缩放因子的 loss 曲线：



从 Loss 曲线可以看出，使用缩放因子时，模型的训练 Loss 和验证 Loss 下降更加平稳，整体收敛过程更稳定；而在不使用缩放因子的情况下，Loss 曲线波动相对更明显，收敛速度也可能变慢。这是因为在注意力计算中，Q 和 K 的点积会随着维度  $d_k$  的增大而变大，如果不进行缩放，softmax 输入值容易过大，从而导致输出分布过于尖锐，进而影响梯度传播的稳定性。加入  $1/\sqrt{d_k}$  后，注意力分数的数值范围得到控制，softmax 不容易饱和，因此训练过程通常更加稳定。

## 5.2 注意力热力图分析

为了观察模型是否学习到了医学症状之间的语义关联，本实验在训练完成后随机抽取一条病历样本，并提取其中一个注意力头的 Attention Map，绘制为热力图。



从注意力热力图中可以看到，模型并不是平均地关注所有症状，而是会对部分关键症状分配更高的注意力权重。这说明模型在训练过程中已经学习到症状之间的相关性，并能够利用这些关系进行疾病分类。例如，如果热力图中“咳嗽”与“痰”“发热”之间存在较深颜色的区域，可以说明模型关注到了呼吸道症状之间的联系；如果“腹泻”与“稀便”“腹胀”等词之间权重较高，则说明模型捕捉到了消化系统症状的聚合特征。

## 六、总结

本实验基于自定义的 **Multi-Head Attention** 模块，完成了一个面向医学症状文本的疾病分类模型设计与实现。实验中首先对病历数据进行了预处理，包括症状序列构建、词表映射、整数编码、序列截断与填充以及 **Padding Mask** 的生成；随后手写实现了 **ScaledDotProductAttention** 和 **MultiHeadAttention**，并在此基础上搭建了包含残差连接、**LayerNorm** 和前馈网络的简易 **Transformer Encoder**，最终实现了疾病分类任务。

从实验结果来看，所构建的模型能够较好地完成测试集上的分类任务，并通过 **Accuracy** 与 **F1-score** 对性能进行了量化评估。同时，通过对“使用缩放因子”和“不使用缩放因子”两种情况进行对比，可以看到缩放操作有助于提高训练过程的稳定性；通过注意力热力图和张量维度追踪图，也进一步验证了多头注意力机制在特征建模和结构分析中的作用。

总体而言，本实验不仅完成了疾病分类模型的搭建，还加深了对注意力机制、**Padding Mask**、多头并行计算以及 **Transformer** 编码器结构的理解。由于本实验采用的是较为简化的单层 **Encoder** 结构，后续仍可以尝试增加网络深度、比较单头与多头注意力差异，或引入更复杂的文本表示方法，以进一步提升模型性能和分析效果。